

第十章 软件架构恢复

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

第10章 软件架构恢复

- 10.1 软件架构恢复概述
 - 10.1.1 软件架构恢复过程
 - 10.1.2 架构信息提取
 - 10.1.3 架构恢复技术
- 小结

10.1 软件架构恢复概述（I）

- 软件架构能为用户提供一个设计层次上的视图，使用户能够更加容易理解系统，并且能够准确定位所需维护的代码和影响的范围。
- 软件架构可以帮助开发人员在保持外部行为不变的前提下，通过改变软件内部结构来增加软件易理解性、可扩展性和可重用性等，进而改进软件质量。
- 然而，往往随着产品演化周期的更迭，文档得不到及时充分的更新，代码跟实际架构之间的偏差越来越大。即出现软件架构腐蚀。

10.1 软件架构恢复概述（2）

- 架构腐蚀会导致软件演化过程中出现工程质量的恶化。
- 若通过人工阅读代码的方式理解架构，费时费力，并非良策。
- 在此情况下，通过逆向工程恢复软件架构就很有意义。

10.1 软件架构恢复概述 (3)

- **逆向工程** (reverse engineering) 被定义为通过对系统的分析, 实现对目标系统由低层到高层的抽象, 描述软件系统的结构、逻辑以及组件之间的相互作用。
- **软件架构恢复** (software architecture recovery) 实际上属于“逆向工程”的研究和实践范畴。

10.1 软件架构恢复概述（4）

- 软件架构恢复的主要目的是从工程项目中**获取所需的架构信息，恢复出架构的组成元素**，即组件元素、连接件元素、架构模式以及架构的配置信息等。
- 通过对恢复出来的软件体系结构进行分析，开发人员可以快速对其进行必要的评估，在软件设计、编码和测试等多个阶段进行相应的改进。

10.1.1 软件架构恢复过程（I）

- 软件架构的恢复过程通常有三种形式
 - 自顶向下：
 - 将架构师所提供的架构设计和描述信息与代码信息相结合，检查架构是否合理，进一步对架构进行精化。（如反射模型）
 - 自底向上
 - 从源代码提取架构视图，然后对架构视图进一步抽象获得最终的架构视图。（最常见的方法）
 - 混合过程
 - 一方面，依据底层的架构信息恢复出架构视图；另一方面，依据高层的架构信息校验恢复出来的架构实体是否存在缺陷等。

10.1.1 软件架构恢复过程 (2)

- 不管采用哪种架构恢复过程，一般可以把恢复过程概括为信息提取和信息表达两部分。
- **信息提取**是指识别架构元素及其之间的依赖关系
 - 静态分析
 - 不运行待分析程序，从文档（包括源码、设计文档、目录结构等）出发对软件进行分析；
 - 动态分析
 - 通过目标系统的一次或多次运行，收集程序间依赖关系，解释系统的运行时行为的分析过程。
- **信息表达**则是在更高的抽象层次上建立软件系统的表达形式。

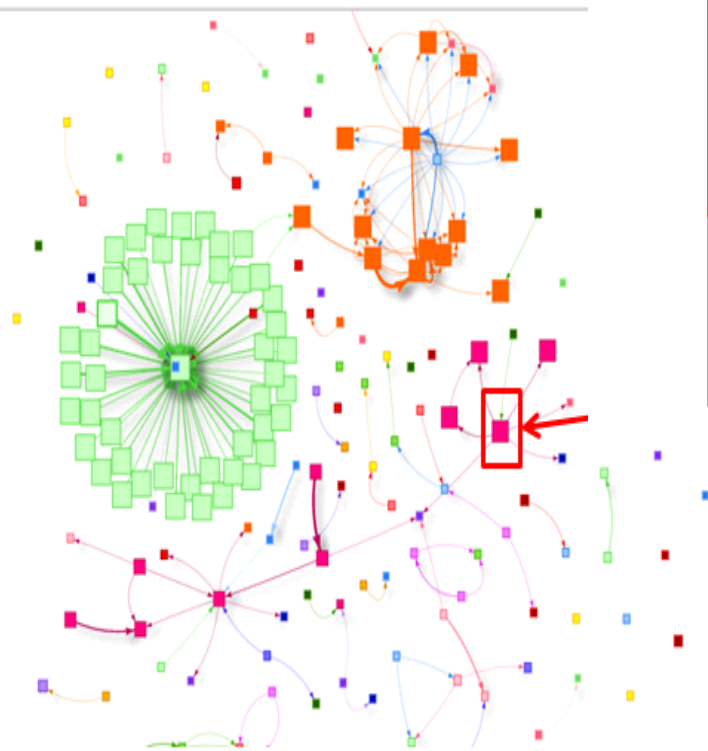
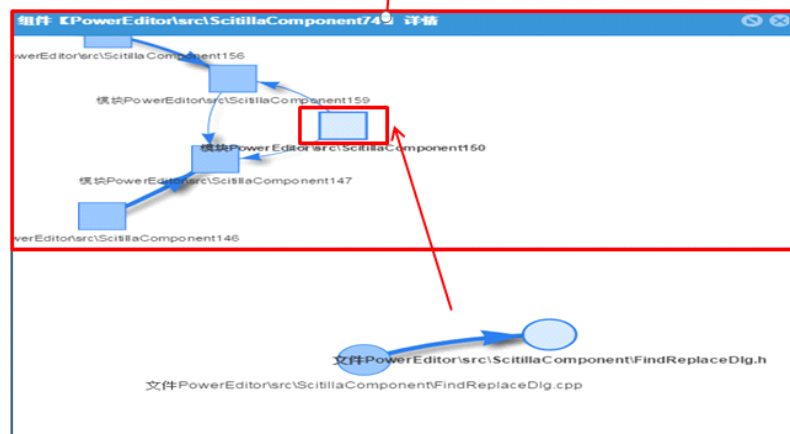
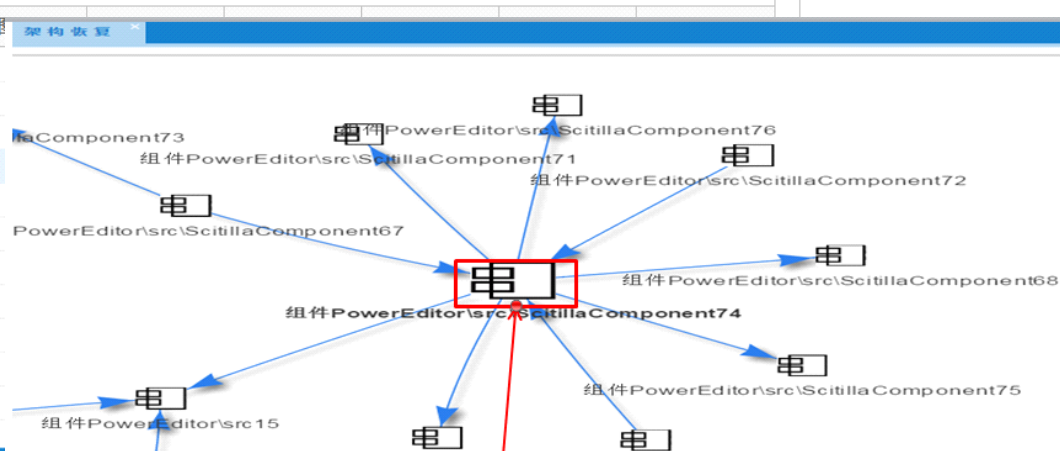
请输入组件名称：

搜索

清空

组件名	有效代码行	圈复杂度	组件图出度	组件图入度	组件图
[-1]0	159532	31390	1	8	4
[-1]1	13893	2782	2	0	1
2	1112	275	1	1	2
builtin3	935	223	1	0	1
sha1dc4	1963	150	0	1	5
vcs-svn5	278	54	1	2	3
6	599	106	2	1	2
vcs-svn7	824	173	3	0	1
refs8	2869	497	1	0	1
compat9	2320	597	1	0	1
contriblcred...	944	201	0	0	1
HEAD111	45	2	0	0	1

架构恢复



10.1.2 软件架构信息提取（I）

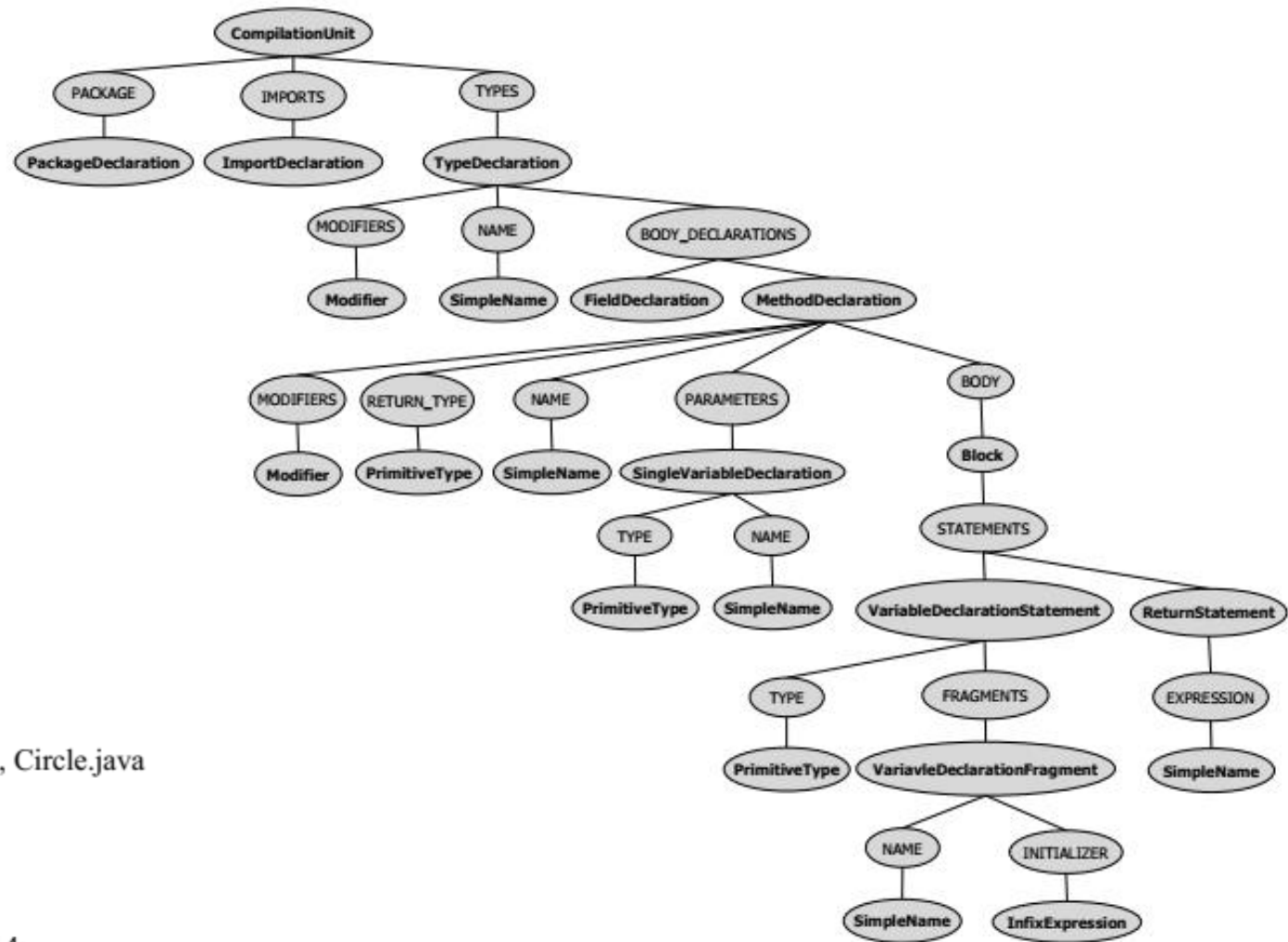
- 静态信息获取方法
 - 基于源代码（Source Code）
 - 基于编译构建过程（Compile Build）
 - 基于数据流（Data Flow）
 - 基于程序切片（Program Slicing）
 - 基于设计文档（Design Document）
 - 基于目录结构（Directory Structure）

10.1.2 软件架构信息提取 (2)

- 基于源代码 (Source Code)
 - 通过构建类似于词法分析和语法分析的平台对程序源码进行文本分析或特征定位，采用信息检索技术发现文档之间的关联。
 - 此类方法仅仅进行源代码扫描，速度快效率高，但如果一个程序存在无意义的标识符，就可能会影响分析结果的精确性。

10.1.2 软件架构信息提取 (3)

- 基于编译构建过程 (Compile Build)
 - 在程序编译执行的平台的基础之上，利用编译的中间结果进行程序依赖分析。
 - 此类方法通过对编译源代码过程中所能得到的中间结果文件（如AST、CFG和RTL文件等）进行解析，提取到程序中的函数调用关系信息
 - 这类信息中可能存在大量的冗余。



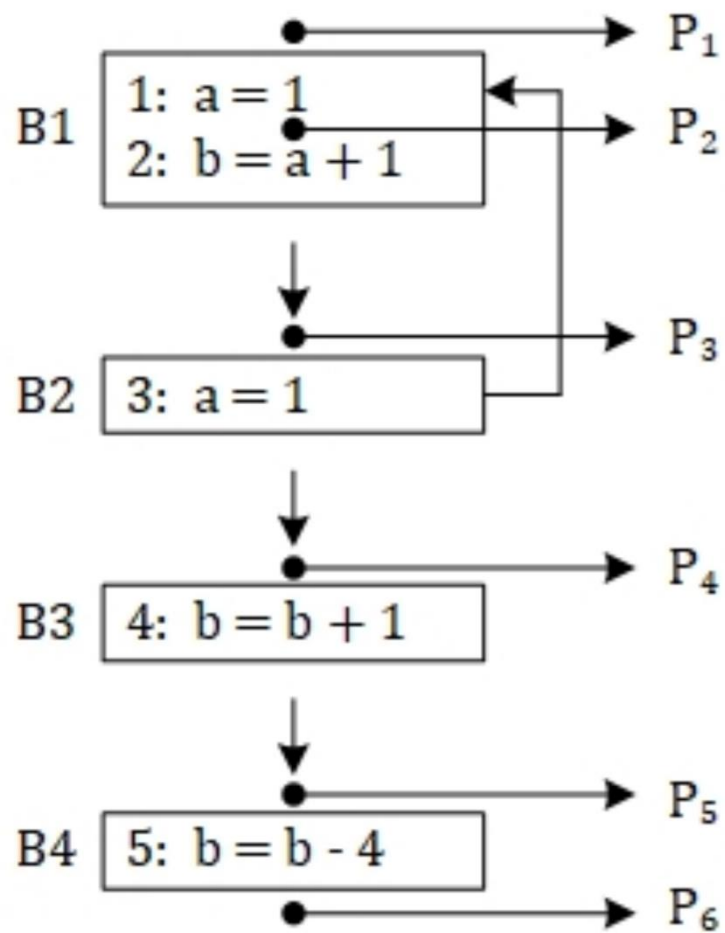
// An example of java program, Circle.java

```

01. package a.b.c;
02. import seu.edu.cn.Shape;
03. public class Circle {
04.     private final double pi = 3.14;
05.     public double getSquire (double r) {
06.         double result = pi*r*r;
07.         return result;
08.     }
09. }
  
```


10.1.2 软件架构信息提取 (4)

- 基于数据流 (Data Flow)
 - 数据流分析是在不执行程序的情况下，收集程序数据的运行时信息，**分析程序中数据对象间的关系**，关注于解决程序中从定义到使用的过程以及数据的使用、定义及依赖关系，对确定系统的逻辑构件及其交互关系很重要。
 - 此类方法往往忽略了控制流依赖的影响，分析不够全面。



CSDN @yan学习日记

10.1.2 软件架构信息提取 (5)

- 基于程序切片 (Program Slicing)
 - 切片技术是将程序简化为和某个特殊计算机相关的语句的技术，可以将关注点缩小在一个较小的范围，而不是关注整个程序。
 - 静态切片技术可以帮助找出“可能影响”一个值的所有语句，通过切片可以得到程序之间乃至语句之间的依赖关系。
 - 当应用于大型系统时，这种方法会导致生成的依赖图过于复杂。

10.1.2 软件架构信息提取（6）

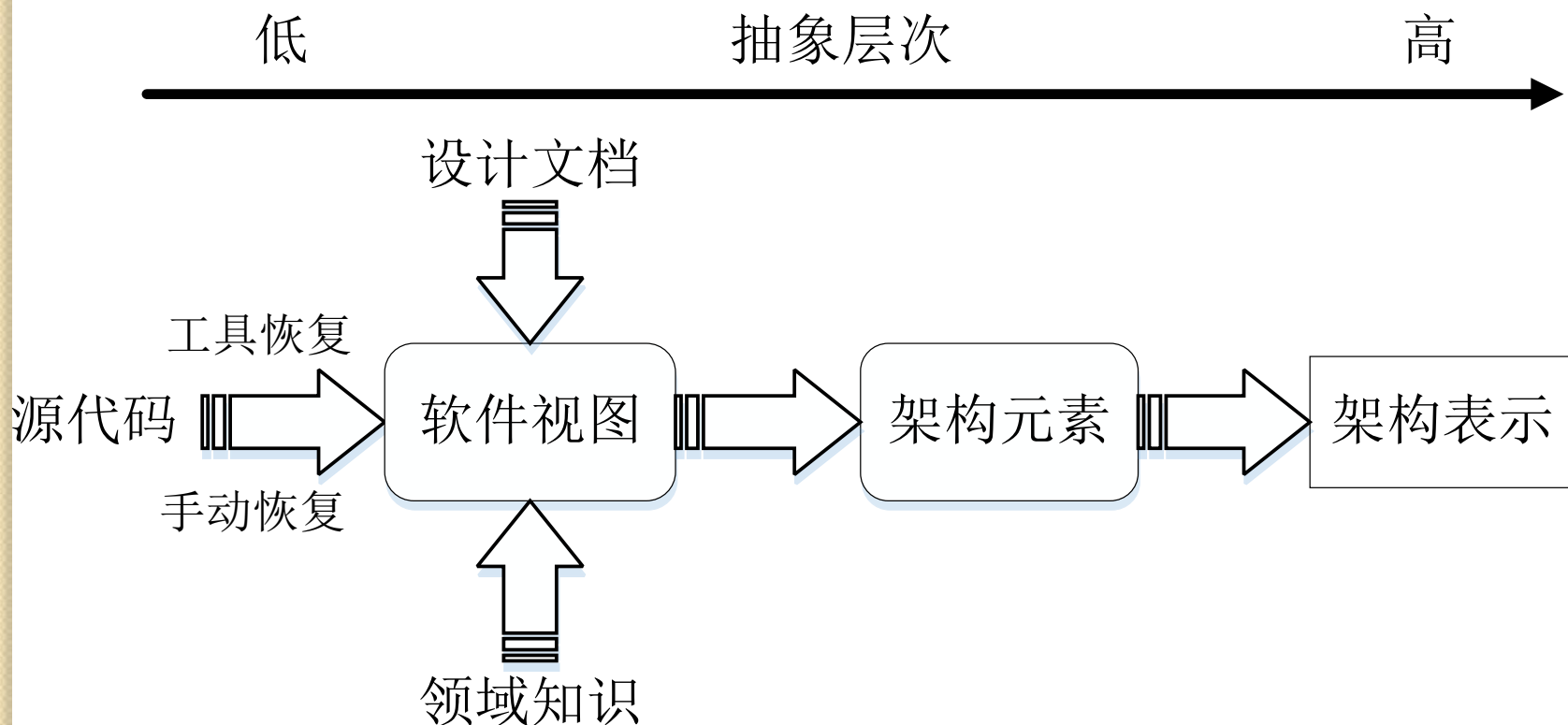
- 基于设计文档（Design Document）
 - 根据可以搜集到的与系统相关的任何文档，例如：系统的设计文档、UML图、代码注释、用户使用手册等，建立对系统的概念和认知，在此基础上获得系统需求、系统服务的商业业务领域、系统提供的服务内容、系统的行为轮廓等方面的信息。

10.1.2 软件架构信息提取 (7)

- 基于目录结构 (Directory Structure)
 - 项目的目录结构不仅与软件项目的设计有关，与开发语言有关，还与软件开发人员的经验有关。
 - 单独从软件的目录结构出发进行架构恢复，信息相对较少，而且并不能反映软件内部原始的依赖关系。
 - 因此，大多数相关研究都是在采取其他方法（如根据设计文档、根据源码等）进行分析的时候，以目录信息作为辅助，帮助构件的划分。相关研究的数量也比较少。

10.1.3 软件架构恢复技术 (I)

- 在架构恢复阶段，其主要目的就是要根据收集到的信息完整地恢复一个系统所覆盖的各种功能。
- 应用架构的恢复策略大致如下图所示：



10.1.3 软件架构恢复技术 (2)

- 主要的架构恢复技术包括：
 - 基于领域知识 (Domain knowledge)
 - 基于聚类 (Clustering)
 - 基于机器学习 (Machine Learning)
 - 基于概念分析 (Concept Analysis)
 - 基于模式匹配 (Pattern Matching)

10.1.3 软件架构恢复技术 (3)

- 基于领域知识 (Domain knowledge)
 - 根据领域知识理解源码或架构设计，并识别出系统中的标准构件。可以采用自顶向下的或者自底向上的过程。
 - 在自顶向下的方法中，通常从设计文档、用户手册等文档性材料出发，整理出软件的架构。
 - 在自底向上的方法中，通常是从代码出发，根据代码中的注释、文件名、变量名的声明等，运用领域知识，恢复架构。
 - 此类方法需要大量人工参与，只适合于小型项目的架构恢复。

10.1.3 软件架构恢复技术（4）

- 基于聚类（Clustering）
 - 自动化软件架构恢复技术中大部分方法都是采用聚类算法来进行，通过对实现级实体（文件、类、函数等）的聚类实现组件的提取。
 - 通常此类方法都是利用数学方法研究和处理给定事物的分类，把一个没有类别标示的样本集按照某种准则划分为若干个子集，将相似的样本尽可能的归为一类，而不相似地样本尽量归为不同类中。

10.1.3 软件架构恢复技术 (5)

- 基于机器学习 (Machine Learning)
 - 从源码中提取实体和特征之后，通过数据训练集对组件识别进行训练，从而得到恢复后的架构。
 - 基于机器学习的方法一般不会单独使用，而是作为聚类算法的补充，用以提高聚类的精度。
 - 训练集的获取比较困难。

10.1.3 软件架构恢复技术 (6)

- 基于概念分析 (Concept Analysis)
 - 基于FCA (Formal Concept Analysis) 从源码中识别出各种概念结构, 将具有共同特征的对象集合起来构成组件。
- 基于模式匹配 (Pattern Matching)
 - 基于一个架构恢复的交互环境, 把恢复过程建模为一个将高层模式图(从专家知识和设计文档得到的架构模式表示)与实体关系图(即源代码系统实体的表示形式)进行匹配的图模式匹配问题。

10.1.3 软件架构恢复技术 (7)

- 国际上最具代表性的自动化软件架构恢复技术包括以下几种：
 - ACDC (Algorithm for Comprehension-Driven Clustering)技术
 - WCA (Weighted Combined Algorithm)技术
 - LIMBO (scaLable InforMation BOttleneck)技术
 - Bunch技术
 - ZBR (Zone-Based Recovery)技术
 - ARC (Architecture Recovery using Concerns)技术

clone	2018.08.29	-
☐ 1.1.1.1		版本详情
开发视图:	文件目录树	
架构视图:	架构恢复 编译构建	添加版本
度量视图:	度量基本信息 架构演进性及原则达成性度量 坏味检测	
重构视图:	变更检测 架构重构 面向持续演进的重构	项目详情
模式识别:	架构模式识别	
aaaa	2018.08.29	+
openbbs	2018.08.30	+

小结

- 软件架构恢复过程
 - 自顶向下，自底向上，混合
 - 信息提取过程&信息表达过程
- 架构信息提取
 - 基于源代码、基于编译构建过程、基于数据流、基于程序切片、基于设计文档、基于目录结构
- 架构恢复技术
 - 基于领域知识、基于聚类、基于机器学习、基于概念分析、基于模式匹配